

AIMLZG546 - Software Engineering for Machine Learning

Assignment I - Report

Group No: 13

Project: Credit Card Fraud Detection

Repository: <https://github.com/sanjaykumarpushadapu/SE-ML-Assessments>

Group Details

Sl. No	BITS ID	Name	Contribution of team member (Qualitative)	Percentage Contribution (out of 100)
1	2025ae05576	Anirudh Anand	Report lead. Wrote the domain and problem statement, ML problem formulation, stakeholders, scope, functional requirements and measurable goals (Sections 1 and 2) and the GR4ML Business View. Compiled the final report and PDF.	25
2	2025ae05178	Aniketh Paul	Wrote the GR4ML Analytics Design View, the Data Preparation View and the top three quality requirements with justification (Sections 3.2, 3.3 and 4). Built the data part of the pipeline in the notebook: EDA, ingest, clean, features and split filters.	25
3	2025ae05898	Pushadapu Sanjay Kumar	Designed the system architecture diagram showing ML and non-ML components, selected the two patterns and wrote Sections 5 and 6. Built the FastAPI prediction service, input validation, prediction logging and the automated tests.	25
4	2025af05111	Biswajeet Mahato	Built the model part of the pipeline: train and evaluate filters, threshold selection on validation, MLflow tracking and model registry. Ran the notebook end to end, took the screenshots and wrote Sections 7.2, 7.3, 8 and 9.	25
			Total	100

Objective 1: Requirements Formulation

1. Domain and Problem Statement

1.1 Domain

The domain we chose is banking, and within it credit card fraud detection. Millions of card payments happen every day and each one has to be approved or declined in a few seconds. If a fraudulent payment goes through, the bank normally has to bear the loss and the customer also loses trust in the bank. So we felt this is a good practical area for a machine learning application.

1.2 Problem Statement

Banks lose a lot of money every year because of fraudulent credit card transactions. The fraud team cannot check every payment by hand, and manual checking is too slow to stop a payment that is being processed. Fraud is also very rare compared to genuine payments (less than 1%), so fixed rules either miss the fraud or block many genuine customers.

In this assignment we propose an application that checks each card transaction when it arrives and gives the probability that it is fraud. Transactions with a high probability are flagged, and the fraud analysts look only at the flagged ones instead of all transactions. Our goal is to catch at least 90% of the frauds, wrongly flag less than 2% of genuine transactions, and give the answer fast enough to be used during the payment.

1.3 Why Machine Learning is Needed

Most banks already have rule based checks, for example blocking a payment above some amount or from an unusual country. Someone has to write these rules by hand, and fraudsters slowly find ways around them. A rule also looks at only one or two fields at a time, and if the rules are made stricter, more genuine payments get declined.

A machine learning model can learn the fraud patterns from old transactions that are already marked as fraud or genuine. It can look at many features together and can be trained again when new fraud patterns come. The model also gives a probability instead of a plain yes or no, so the bank can pick a threshold that gives an acceptable balance between missed frauds and false alarms.

1.4 Stakeholders

Stakeholder	Interest in the system
Bank risk manager	Reduce fraud losses without increasing complaints from customers
Fraud analyst	Get a reliable score so that only suspicious transactions need manual review
Card holder (customer)	Genuine payments should not be declined
ML and development team	Build and maintain a model and service that are accurate and fast

1.5 Scope

These are inside the scope of this assignment:

- Checking a single transaction and returning a fraud label with its probability
- Providing this check through a prediction service (API)
- Training the model and storing it in a model registry
- Logging every prediction made by the service

Outside the scope are: connecting to a real payment network, sending alerts to customers, handling chargebacks and using real customer card data. We use a public anonymised dataset instead.

1.6 Formulation as a Machine Learning Problem

We treat this as a binary classification problem. Each transaction is labelled fraud (1) or genuine (0). The model takes the transaction features as input and gives the probability of fraud, and the final label is

decided by a threshold.

We use the "Credit Card Fraud Detection" dataset from Kaggle. It has card transactions made by European card holders, with 284,807 transactions of which only 492 (0.173%) are fraud. Except for Time and Amount, the features V1 to V28 are already anonymised by the dataset provider. The two classes are very unbalanced, so accuracy alone does not tell us much (a model that says everything is genuine would still be 99.8% accurate). Because of this we judge the model with recall, precision, F1 score, the area under the precision-recall curve and the confusion matrix.

For the design we picked two architectural patterns: Pipe-and-Filter for the training pipeline, and Microservices for serving the model as a separate prediction service together with a model registry. They are explained in Section 6.

1.7 Data Exploration (genuine vs fraud)

The charts below are copied from the notebook outputs (Section 2, Exploratory data analysis) by `build_report.py`. They show how rare fraud is and in which columns fraud looks different from genuine transactions.

Figure 1 - Class balance and amount. Only 492 of the 284,807 transactions (0.173%) are fraud, and most amounts are small with a long tail of large payments.

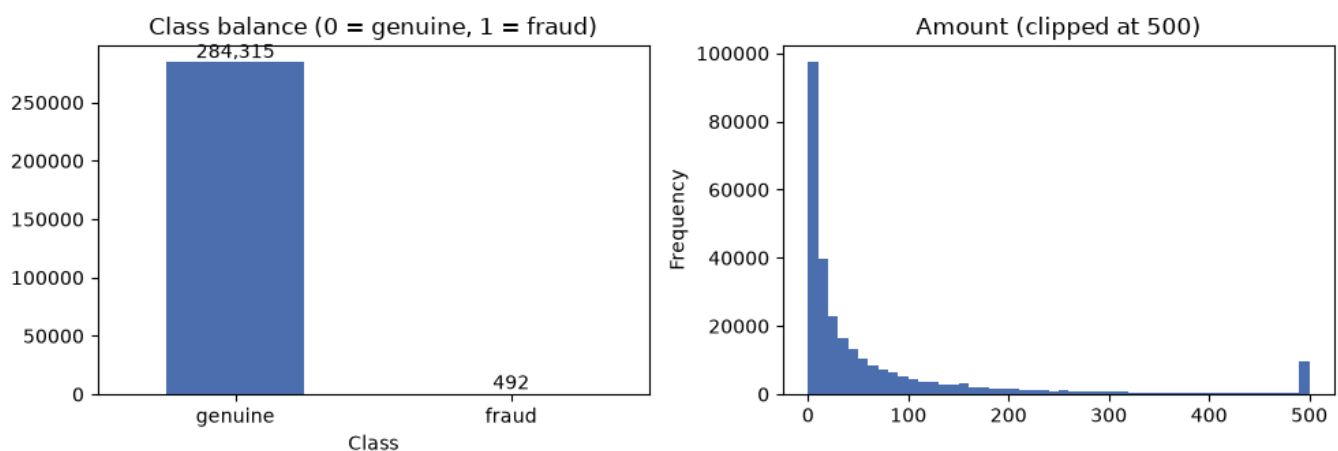


Figure 2 - Time of the transactions. Each bar is the share of that class's transactions in one hour (Time is counted from the first transaction, so hour 0 is not midnight). 27.0% of the frauds fall in hours 0

to 6, where only 9.8% of the genuine transactions happen, so fraud is relatively more common when there are few genuine payments. This is why Time is kept as a model input.

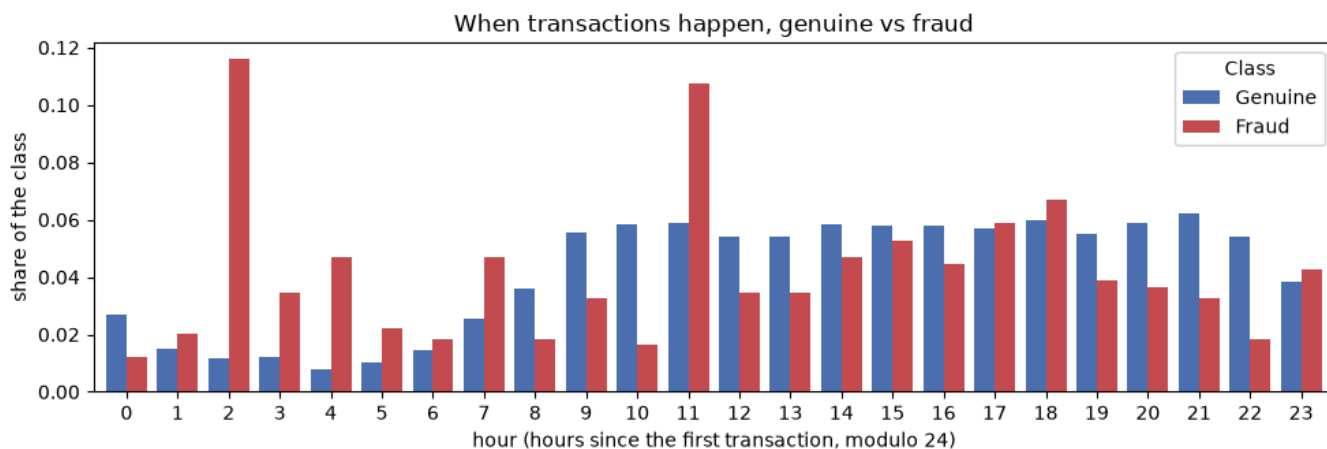


Figure 3 - Amount. The median fraud is 9.25 against 22.00 for a genuine payment, and the 95th percentile is 640.90 against 364.41, so frauds are more spread out. Amount has a long tail, with a maximum of 25,691.16, which is why it is scaled with a RobustScaler in the Data Preparation View (Section 3.3).

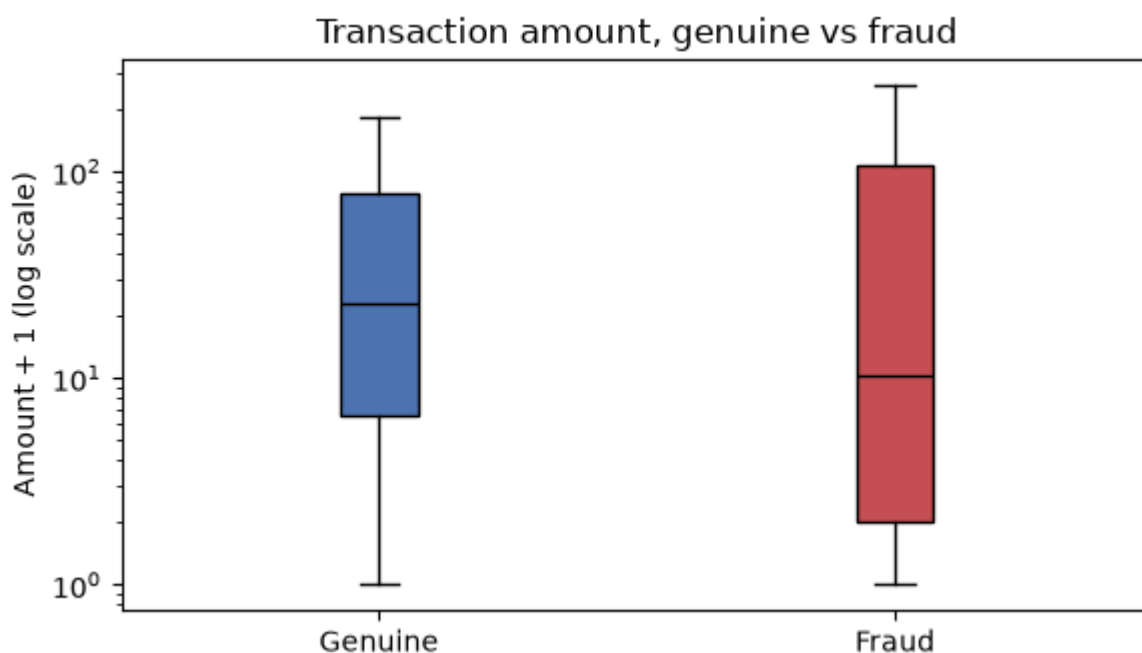
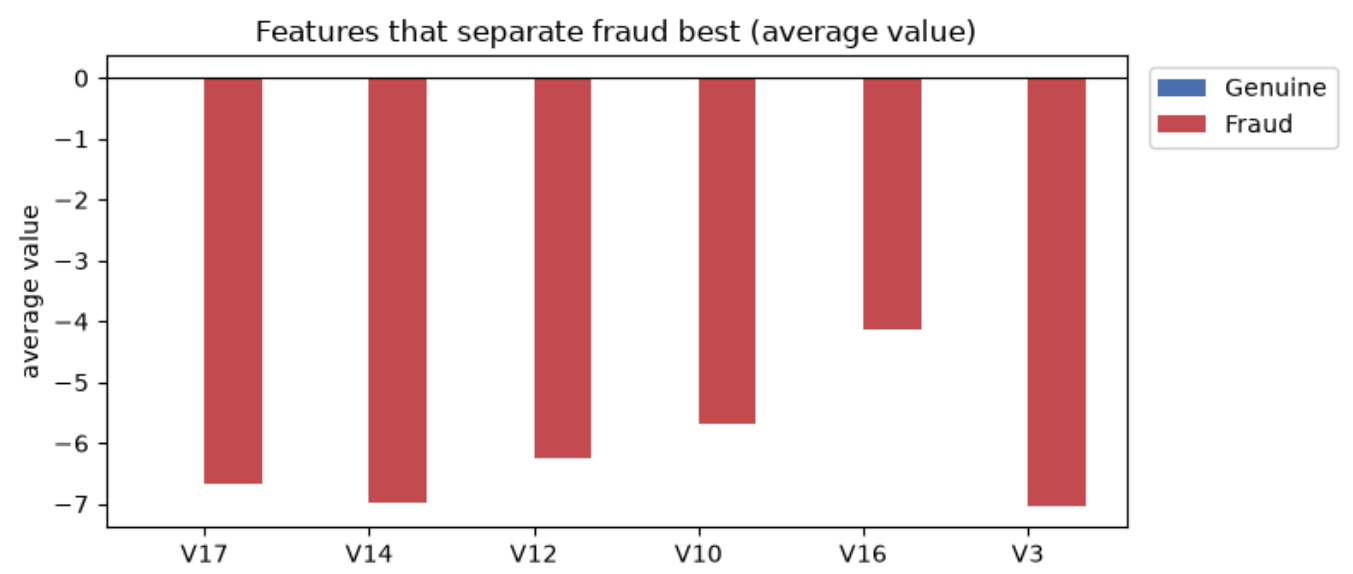


Figure 4 - Features that separate fraud best. These are the 6 V features whose average differs most between the classes, measured in standard deviations (V17 7.9, V14 7.3, V12 6.3, V10 5.2, V16 4.7, V3 4.7). The genuine averages are close to 0 and the fraud averages are far from it. A simple linear model

can use such differences, which supports Logistic Regression as a candidate in the Analytics Design View (Section 3.2).



2. Requirement Specifications and Measurable Goals

2.1 Functional requirements

Each requirement is checked by an automated test (named after the business rule in Section 2.3) or by a step of the notebook run, so it can be verified and not only stated.

ID	Requirement	Verified by
FR1	The system accepts one card transaction (Time, V1 to V28 and Amount) through a POST /predict API call.	test_BR001_label_is_probability_against_threshold; fraud and genuine test calls (Screenshots 4 and 5)
FR2	The system returns the fraud probability, the fraud label, the threshold used and the model version.	test_BR001_boundary_probability_equal_to_threshold_is_fraud, test_BR009_response_carries_model_version
FR3	The system rejects invalid input (missing field, wrong type, NaN or infinity, negative Amount, unknown field) with HTTP 422 and does not return a prediction for it.	test_BR007_missing_field_rejected, ..._negative_amount_rejected, ..._wrong_type_rejected, ..._extra_field_rejected, ..._nan_and_inf_rejected; invalid-request cell of the notebook
FR4	The system writes one row to a prediction log for every prediction made.	test_BR008_every_call_is_logged, test_BR008_log_failure_is_an_error_not_silent, test_BR007_invalid_input_is_not_logged; Screenshot 6
FR5	The training pipeline reads the dataset, cleans it, splits it, trains the model and reports the evaluation metrics. The pipeline can be run again to retrain.	test_BR010_run_pipeline_chains_filters_in_order, test_BR005_metrics_include_required_measures, test_BR011_same_seed_gives_same_split_and_results; Screenshot 1
FR6	The trained model is stored in a model registry, and the prediction service loads only the registered	test_BR009_loads_only_the_champion_alias, test_BR009_tracking_uri_can_point_to_registry_service; Screenshots 2 and 7

ID	Requirement	Verified by
	model marked as champion.	
FR7	The service has a health endpoint that reports whether it is running and which model version it serves.	test_BR009_response_carries_model_version (calls /health); Docker Compose health checks (Screenshot 7)

2.2 Measurable goals

ID	Goal	GR4ML concept	Metric	Target	Result (latest run)	Met?
G1	Catch fraud	Indicator of the strategic goal; softgoal High recall	Recall on the test set	at least 90%	85.3%	No
G2	Avoid blocking genuine customers	Indicator of the strategic goal; softgoal Few false alarms	False-positive rate on the test set	below 2%	1.08%	Yes
G3	Decide while the payment is processed	Softgoal Low latency	p95 latency of the prediction API	below 200 ms	32.0 ms	Yes
G4	Reduce losses	Strategic goal	Share of fraud money caught (test set)	at least 30%	82.4%	Yes

Model-quality results use the verified registry evidence identified in Section 7; latency uses the saved native API measurements. Historical notebook outputs are labelled separately. If a target is not met, we report the real value and do not tune anything on the test set.

Acceptance status: G1 fails: test recall is 85.26%, below the 90% target. This is an assignment demonstrator, not an accepted production payment service. Passing software tests does not establish model-quality acceptance. The target and held-out results remain unchanged.

The held-out evaluation caught 81 of 95 fraud cases. The 95% exact binomial confidence interval for recall is approximately 76.51% to 91.70%, reflecting uncertainty from the small fraud sample. This interval does not make G1 pass. Test-set threshold trade-offs are post-hoc analysis only, not inputs to model selection or promotion.

To improve recall, investigate candidate models, class weighting and threshold selection using training and validation data only. Freeze the selected pipeline and acceptance criteria before evaluating on fresh, untouched hold-out data. The already-inspected test set must not become a threshold-tuning or model-selection set.

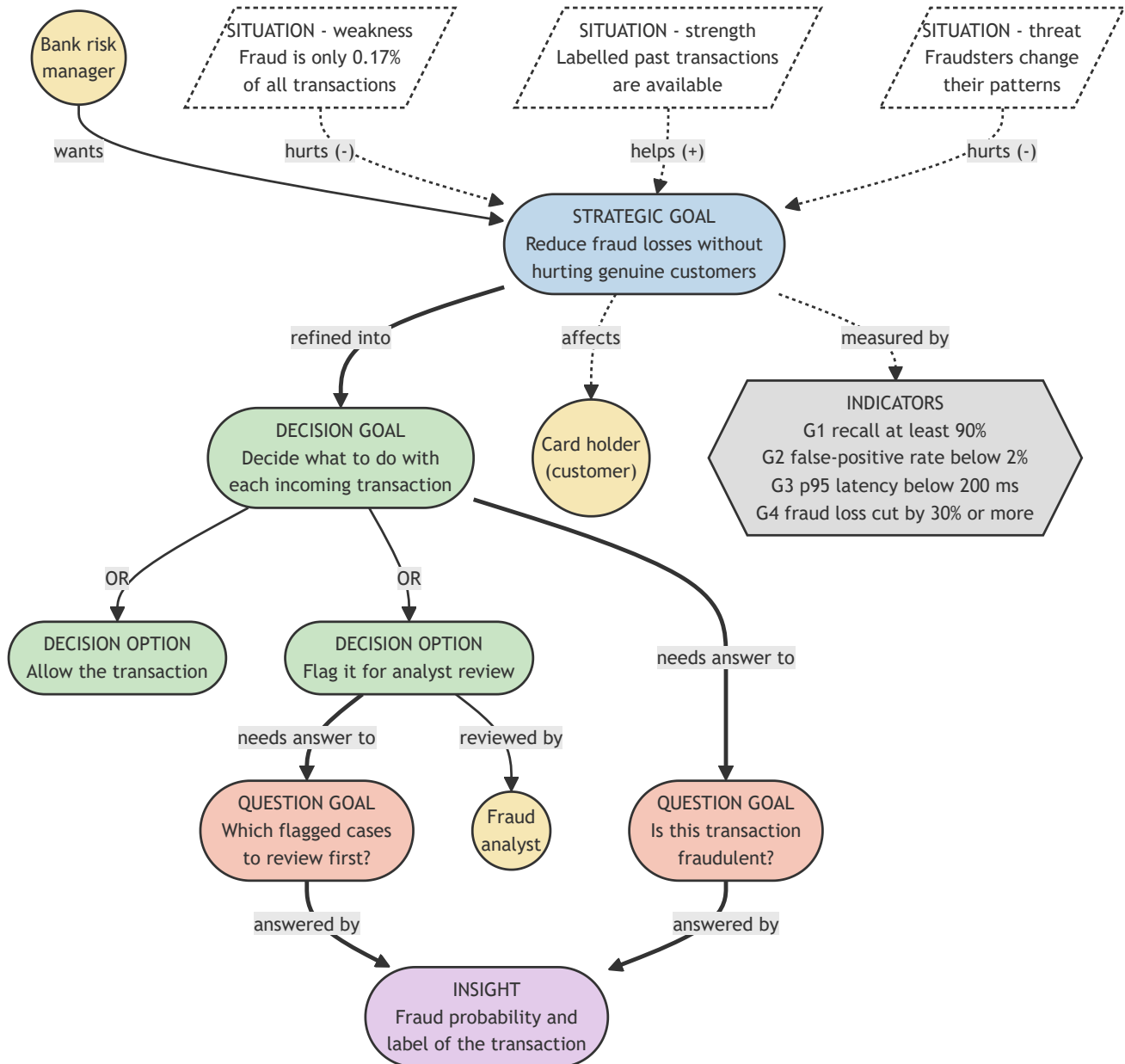
2.3 Business rules

ID	Rule
BR-001	A transaction is labelled fraud only if its fraud probability is greater than or equal to the threshold.
BR-002	The threshold is chosen on the validation set (highest value with recall at least 0.90), stored as a run parameter and kept in one config place.
BR-003	The scaler is fitted on the training data only.

ID	Rule
BR-004	The scaling is saved inside the model pipeline and applied the same way at serving time.
BR-005	Evaluation reports recall, precision, F1, PR-AUC, ROC-AUC, false-positive rate and the confusion matrix.
BR-006	Targets are compared honestly with the test results; nothing is tuned on the test set.
BR-007	Invalid input is rejected with HTTP 422 and is not logged.
BR-008	One log row is written per prediction; a failed write is reported as an error.
BR-009	The API loads only the model with the alias champion.
BR-010	Training runs as a chain of six filters: ingest, clean, features, split, train, evaluate.
BR-011	A fixed seed (42) and a single config make the run reproducible.

3. GR4ML Views

3.1 Business View

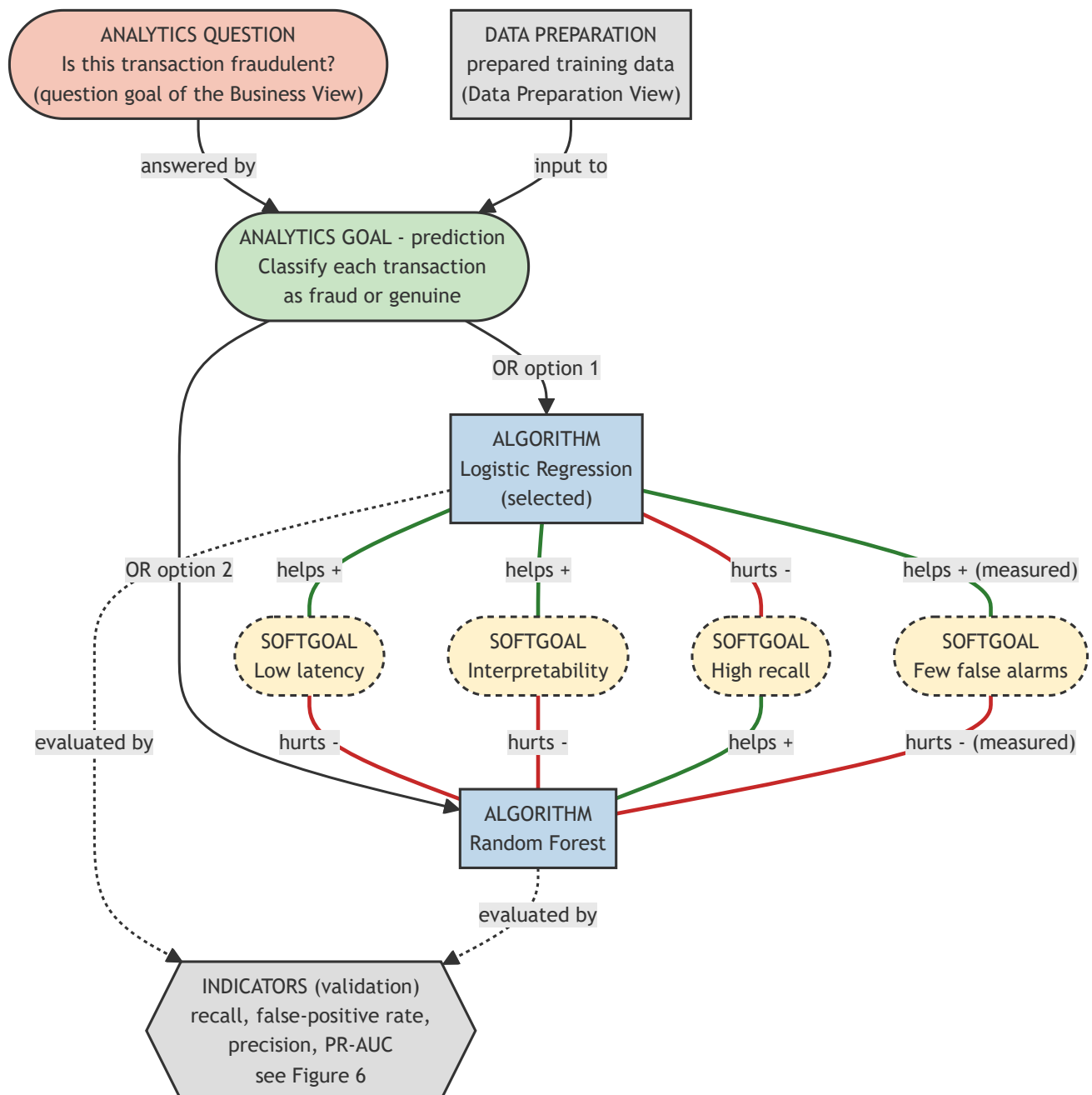


Colour key: yellow circles are actors. Blue, green, red and purple rounded boxes are the strategic goal, the decision goal with its options, the question goals and the insight. The grey hexagon holds the indicators, and the dashed white parallelograms are situations. Thick arrows connect the goals, thin arrows show OR alternatives and who acts, and dotted lines are influence links (helps +, hurts -), "affects" and "measured by".

Explanation: The bank risk manager wants the strategic goal, which is to reduce fraud losses without hurting genuine customers. The card holder is affected because a wrongly blocked payment is annoying for them. The strategic goal is refined into one decision goal: decide what to do with each incoming transaction. This decision has two alternatives (an OR refinement): allow the transaction, or flag it for review by a fraud analyst. To take the decision the bank needs the answer to the question goal "is this transaction fraudulent?", and for the flagged cases the analyst also needs to know which ones to review

first. Both questions are answered by the same insight, the fraud probability and label of the transaction (a higher probability is reviewed first). Three situations influence the strategic goal. Fraud is very rare (weakness), so it is hard to learn and easy to miss. Labelled past transactions exist (strength), so a supervised model can be trained. Fraudsters change their patterns (threat), so the model must be retrained, which the pipeline and the registry make possible. The strategic goal is measured by the four indicators G1 to G4, which are the measurable goals in Section 2.2.

3.2 Analytics Design View

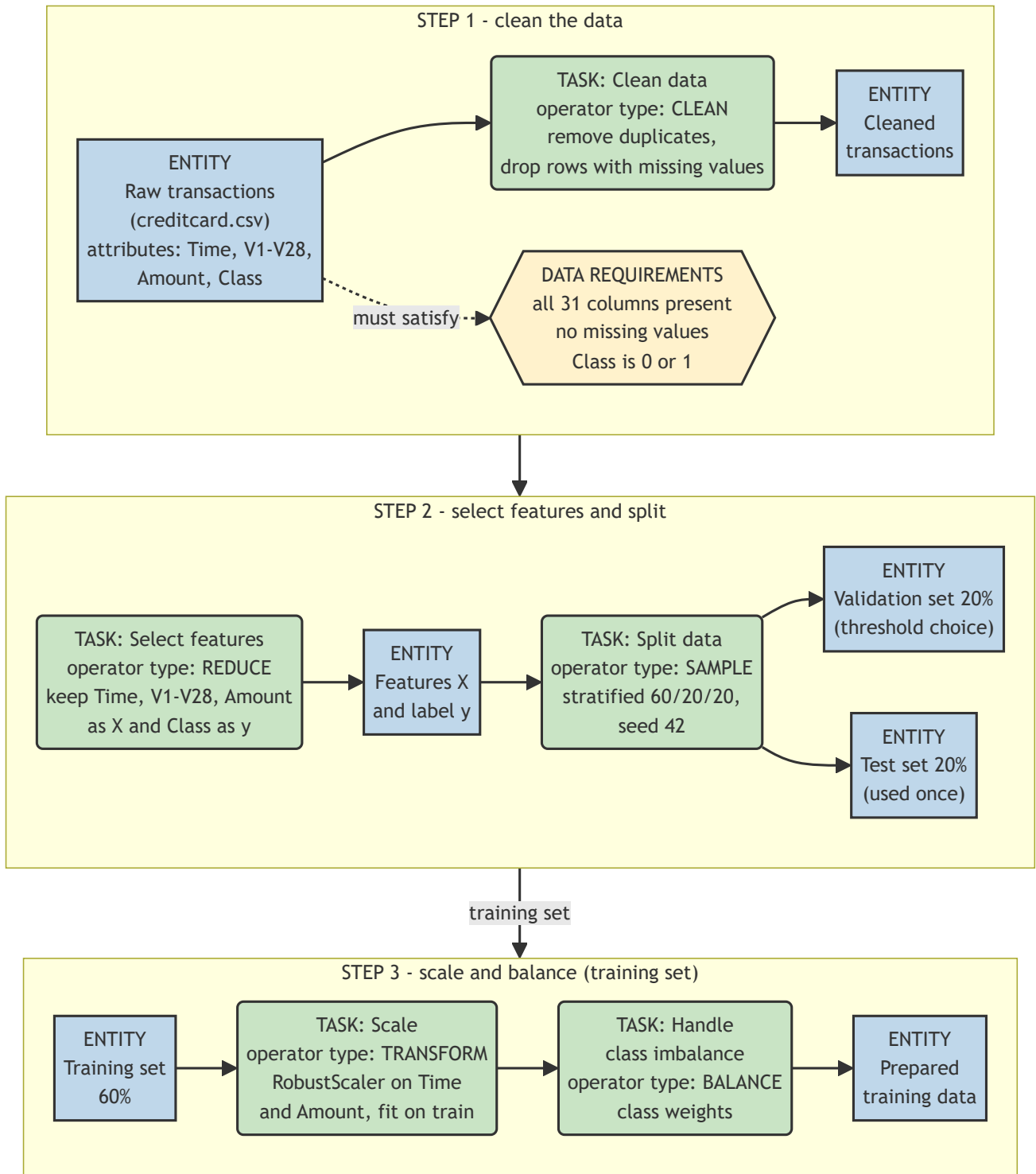


Colour key: green links are positive contributions (+) and red links are negative contributions (-). Links marked "measured" come from our validation results (Figure 6). The others are what we expected from the course material before training.

Explanation: The analytics question comes from the Business View: is this transaction fraudulent? It is answered by an analytics goal of type prediction, a binary classification of each transaction. We

considered two algorithms as OR alternatives and implemented both: Logistic Regression and Random Forest. Four softgoals judge them. We expected Logistic Regression to help low latency and interpretability but to hurt high recall, and Random Forest to help recall but to hurt latency and interpretability. For few false alarms we use the measured result: with both thresholds set to reach 90% validation recall, Logistic Regression flagged far fewer genuine transactions than Random Forest, so this softgoal decided the choice. Both algorithms are evaluated with the same indicators on the validation set (recall, false-positive rate, precision and PR-AUC), and both are trained on the prepared data described in the Data Preparation View. In our run Logistic Regression was selected because it had the lower validation false-positive rate (1.02% against 4.68% for Random Forest), although Random Forest had the better PR-AUC (0.859 against 0.786). The recall of Logistic Regression was 90.4% on validation and 85.3% on the test set.

3.3 Data Preparation View



Colour key: blue boxes are entities (data) with their attributes, green boxes are data preparation tasks, and each task names its GR4ML operator type (CLEAN, REDUCE, SAMPLE, TRANSFORM, BALANCE) and how we applied it. The yellow hexagon lists the data requirements the raw data must meet. Arrows show the flow of the data.

Explanation: The raw entity has Time, V1 to V28, Amount and Class; ingest requires all 31 columns. The CLEAN task drops 1,081 duplicate rows and rows with missing values. It then rejects an empty dataset, nonnumeric or infinite features, negative Amount, and labels other than 0 or 1; both classes must remain. The REDUCE task selects the 30 inputs and label. The SAMPLE task uses a stratified

60/20/20 split with seed 42. On training data only, TRANSFORM fits the RobustScaler for Time and Amount, and BALANCE supplies class weights to the classifier. Validation selects the threshold; the frozen pipeline is evaluated on the test set. The API also enforces finite features and nonnegative Amount and rejects extra fields.

3.4 GR4ML concept mapping

GR4ML concept	In this project
Actors	Bank risk manager, fraud analyst, card holder
Strategic goal	Reduce fraud losses without hurting genuine customers
Decision goal	Decide what to do with each transaction (OR: allow, or flag for analyst review)
Question goals	Is this transaction fraudulent? Which flagged cases to review first?
Insight	Fraud probability and label per transaction
Situations	Fraud is rare (weakness), labelled history exists (strength), fraud patterns change (threat)
Indicators	G1 recall, G2 false-positive rate, G3 p95 latency, G4 fraud loss reduction (Section 2.2)
Analytics question / goal	Is this transaction fraudulent? / prediction (binary classification)
Algorithms	Logistic Regression (selected), Random Forest
Softgoals	High recall, few false alarms, low latency, interpretability
Data preparation entities	Raw transactions, cleaned data, features and label, train/validation/test sets, prepared data
Tasks and operator types	Clean (CLEAN), select features (REDUCE), split (SAMPLE), scale (TRANSFORM), class weights (BALANCE)
Data requirements	All 31 columns present, no missing values, Class is 0 or 1

4. Top Three Quality Requirements

#	Quality requirement	Measurable target	Justification (why it matters here)	How architecture supports it
1	Accuracy (recall)	Recall at least 90% with false-positive rate below 2%	A fraud that is not detected is a direct loss for the bank. The classes are very unbalanced, so overall accuracy would hide the missed frauds.	We use class weights and a separate validation set to choose the threshold, and we report recall, precision, F1, PR-AUC and the confusion matrix instead of accuracy.
2	Performance (latency)	p95 latency below 200 ms	The decision has to be taken while the payment is being processed, so a late answer is as bad as no answer.	The model is loaded once when the service starts and kept in memory. The prediction service is separate from training, so for each request it only validates the input and calls the model once.
3	Security and privacy	No card number accepted or stored; invalid input rejected	Card data is sensitive, and a service that accepts any input can be misused.	The API accepts only the numeric model features (no card number or name) and rejects any extra field. The prediction log stores only the probability, label, latency and model version.

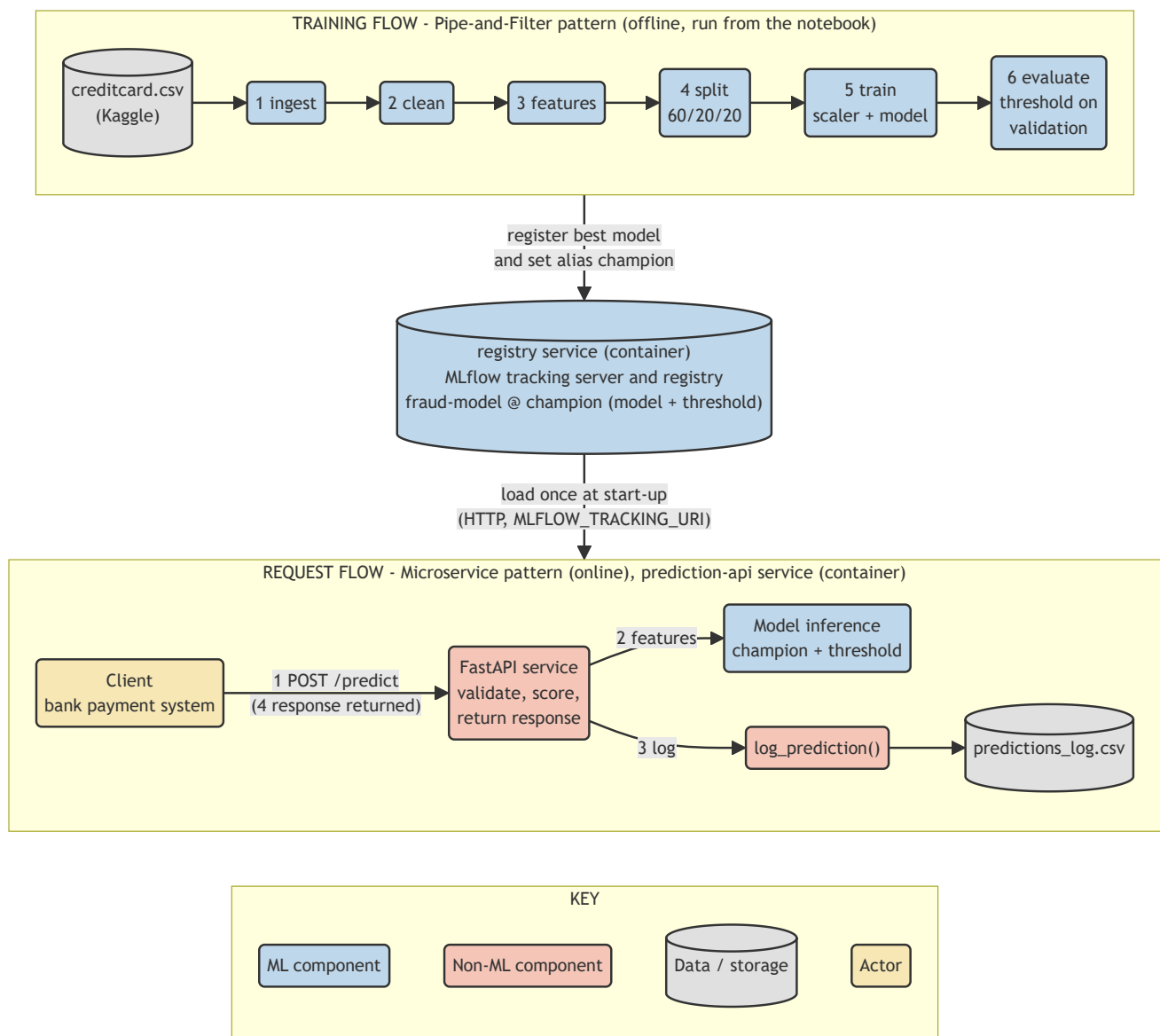
Why these three: each one is tied to a stakeholder and a measurable goal. Recall protects the bank's money (risk manager, G1 and G4), latency is needed because the answer must arrive while the payment is being processed (payment system, G3), and security and privacy protect the card holder's data. We also considered other quality requirements and ranked them lower for this application:

- **Explainability:** useful for fraud analysts, but V1 to V28 are anonymised components, so a feature-level explanation would not mean much to an analyst. We kept interpretability as a softgoal in the Analytics Design View instead.
- **Fairness:** it cannot be measured here, because the dataset has no customer attributes such as age or gender.
- **Availability and scalability:** important in production, but our prototype runs one instance of each service. The microservice design (Section 6.2) allows the prediction service to be scaled separately later.
- **Maintainability:** supported by the Pipe-and-Filter design and the 30+ automated tests, but it does not directly affect the bank's losses or the customer, so it is not in the top three.

Authentication of API clients is not part of this assignment. We list it as a limitation of our prototype.

Objective 2: System Architecture

5. System Architecture Diagram



The key inside the diagram explains the colours: blue is an ML component, red is a non-ML component, grey is data or storage and yellow is an actor.

ML components: the six pipeline filters (ingest, clean, features, split, train, evaluate), the MLflow model registry that keeps the trained model and its threshold, and the model inference step inside the prediction service.

Non-ML components: the FastAPI service with the /predict and /health endpoints, the pydantic request validation, the log_prediction function with the prediction log file, and the client that sends the transactions.

Request flow: (1) The client sends a transaction to /predict. (2) The FastAPI service validates it and gives the features to the loaded champion model, which returns a probability. (3) The service compares the probability with the threshold and writes one row to the prediction log through log_prediction(), and (4) the response goes back to the client.

Deployment: with Docker Compose the registry and the prediction API run as two separate containers that talk over HTTP (Section 7.7).

Training flow: the Kaggle dataset goes through the six filters. In the last filter the threshold is chosen on the validation set, the results are logged to MLflow, and the best model is registered as fraud-model with the alias champion. The prediction service loads this model when it starts.

6. Architectural Patterns

6.1 Pattern 1: Pipe-and-Filter

- Problem: Training a model has several processing steps. If everything is in one long script, it is hard to test a single step or to replace one, for example the model.
- Solution: We divide the processing into independent filters. Each filter takes an input, gives an output and keeps no hidden state, and the output of one filter is the input of the next one.
- How we applied it: The training pipeline has six filters (ingest, clean, features, split, train, evaluate), each in its own module under pipeline/. The file run_pipeline.py only connects them. The train filter accepts a classifier name, so the other filters need no change, and we used this to train both Logistic Regression and Random Forest.
- Benefits and trade-offs: Each filter can be tested alone (the notebook has tests for this) and the order of the steps is easy to follow. The trade-off is that the data is passed between filters as whole tables in memory. That is fine for this dataset, but a much bigger dataset would need streaming.

6.2 Pattern 2: Microservices

- Problem: The payment system of the bank needs the model at any time, while training is a separate job that runs only sometimes and needs far more resources.
- Solution: We make the prediction a small independent service with its own interface and keep it apart from the training code. Managing the models is done by a third component, the registry.
- How we applied it: The FastAPI service in api/main.py only loads the registered model fraud-model with the alias champion and answers /predict and /health. It never trains. MLflow works as the registry between training and serving, so a new model can be registered and picked up

without changing the service code. With Docker Compose the two services run in their own containers: registry (MLflow tracking server, port 5000) and prediction-api (FastAPI, port 8000).

The API reaches the registry only through its HTTP interface

(`MLFLOW_TRACKING_URI=http://registry:5000`), each service has its own health check, and the API starts only when the registry is healthy (Section 7.7, Screenshot 7).

- **Benefits and trade-offs:** The service can be restarted or scaled without touching the training, and every response carries the model version. The trade-off is that there are more parts (service, registry and log) than in a single script, and the service has to be restarted to load a newly registered champion. In our prototype both containers also share the project folder, because the registry keeps the model files there. A production setup would use a shared artifact store (for example object storage) instead.

7. Implementation

7.1 Technology stack

Layer	Technology	Purpose
ML pipeline	Python, pandas, scikit-learn	Filters for data preparation, training and evaluation
Model registry	MLflow (SQLite backend)	Tracking of runs and registry of the champion model
Prediction service	FastAPI, pydantic, uvicorn	Separate prediction microservice with input validation
Logging	CSV file written by <code>log_prediction()</code>	One row per prediction
Testing	pytest, FastAPI TestClient	Tests named after the business rules
Deployment	Docker, Docker Compose	Registry and prediction API as two containers
Runtime	Jupyter notebook 13.ipynb (runs in Google Colab or VS Code)	Runs everything from top to bottom, no GPU needed

7.2 Pipe-and-Filter pipeline

The notebook writes each filter to its own file and `run_pipeline.py` joins them. `ingest` checks that the file and the columns exist, and stops with an error if they do not. `clean` drops duplicate rows and rows with missing values. `features` takes `Time`, `V1` to `V28` and `Amount` as inputs and `Class` as the label. `split` does a stratified 60/20/20 split with seed 42. `train` fits a scikit-learn Pipeline where a `RobustScaler` for `Time` and `Amount` comes before the classifier, so the scaler is fitted on the training set only and is saved together with the model. `evaluate` picks the highest threshold that gives a recall of at least 0.90 on the validation set, and the test set is used only once at the end with this threshold.

```

# each stage hands its output to the next one
raw = ingest(data_path)
df = clean(raw)
X, y = build_features(df)
parts = split(X, y)
# train every candidate on the training set and judge it on the validation set
fitted = {}
for name in candidates:
    model = train(parts["X_train"], parts["y_train"], name)
    threshold, val_m = evaluate_validation(model, parts["X_val"], parts["y_val"])
    fitted[name] = (model, threshold, val_m)
# the winner is chosen from validation results only; the test set is used once at the end
winner = pick_winner(fitted) # lowest validation false-positive rate, PR-AUC breaks ties
model, threshold, val_m = fitted[winner]
test_m = evaluate_test(model, parts["X_test"], parts["y_test"], threshold)

```

Run evidence: These metrics describe Docker champion version 11. Docker screenshots describe the separate Docker champion version 11. The saved native prediction examples serve version 1; latency measurements and charts come from the saved native notebook run. Native and Docker registries have independent version numbers.

Evidence source: Saved Docker notebook version 6 is historical evidence. The current version, threshold, selected model, and evaluation metrics were verified against the live Docker API and champion's MLflow run; saved notebook outputs are historical evidence.

Model results: The dataset has 284,807 rows with 492 frauds (0.173%). After removing 1,081 duplicate rows, 283,726 rows remain with 473 frauds (0.167%). The stratified split gives 170,235 training, 56,745 validation and 56,746 test rows. For each model the threshold is the highest one that still gives a validation recall of at least 0.90.

Model (validation)	Threshold	Recall	Precision	FPR	PR-AUC
Logistic Regression	0.7692	0.9043	0.1286	1.02%	0.786
Random Forest	0.0107	0.9043	0.0310	4.68%	0.859

Both models were tuned to the recall target on validation. Random Forest gives 4.68% false positives on validation, which is above the 2% limit, but Logistic Regression stays within it, so we selected Logistic Regression (version 11 in the registry). Below are the results of the selected model on the test set, which we used only once:

Recall	Precision	F1	PR-AUC	ROC-AUC	FPR
0.8526	0.1169	0.2056	0.6884	0.9644	1.08%

Confusion matrix on the test set: TN 56,039, FP 612, FN 14, TP 81. **The FPR target (below 2%) is met, but the recall target (at least 90%) is not met on the test set (85.3%).** On validation the recall was 90.4%. The test set has only 95 frauds and the validation set 94, so missing just one more fraud changes recall by about one percentage point, and a threshold that is only just enough on validation can fall short on new data. We report the result as it is and did not tune the threshold on the test set.

Fraud loss (goal G4): We measured the loss in money using the Amount column and assumed that every flagged fraud is stopped. The 95 frauds in the test set add up to 14,766.31 and the model catches 12,163.97 of it, so the fraud loss goes down by 82.4%, against the target of 30% (met). The cost is that

genuine transactions worth 205,692.94 were also flagged, and a real bank would have to consider the cost of these blocked payments too. This is only an estimate from the test set and it does not include chargeback fees or the cost of reviewing the flagged transactions.

Note on the selection rule: In our first run we chose the winner by the best validation PR-AUC and Random Forest won. On the test set it missed both targets (recall 87.4%, FPR 4.74%). After seeing this we changed the rule to the lowest validation FPR, with PR-AUC as tie-break, because the false-alarm limit is one of our stated goals. The change was based on validation results only, but we made it after seeing the first run, so we mention it here.

Figure 5 - Confusion matrix on the test set. It shows the four counts given above: 81 of the 95 frauds are caught and 612 genuine transactions are flagged.

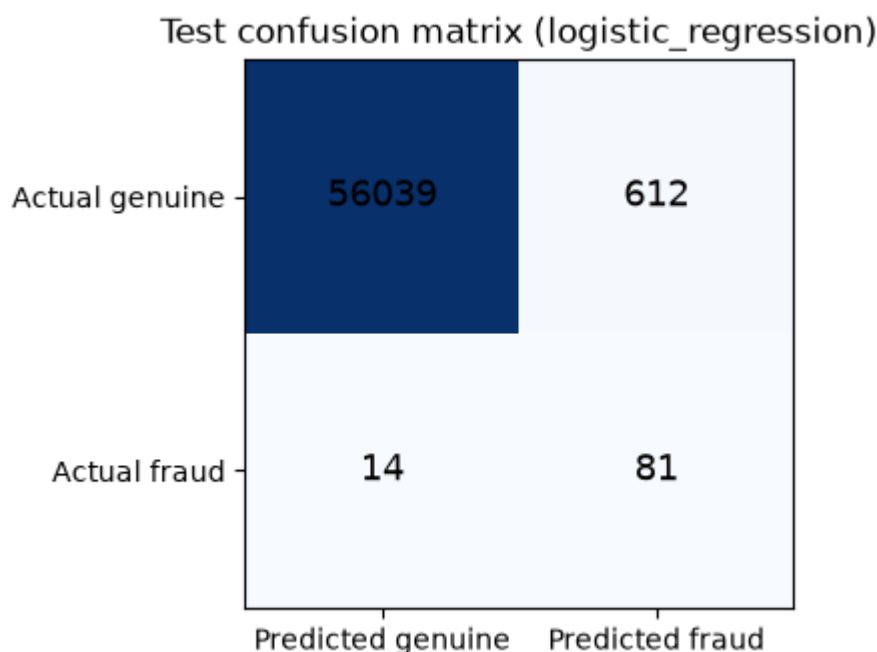


Figure 6 - Model comparison on the validation set. Each threshold was set to reach at least 90% recall on validation, so the false-positive rate decides: Logistic Regression 1.0% (PR-AUC 0.786), Random Forest 4.7% (PR-AUC 0.859), against the 2% target line. Logistic Regression has the lowest false-positive rate and was selected.

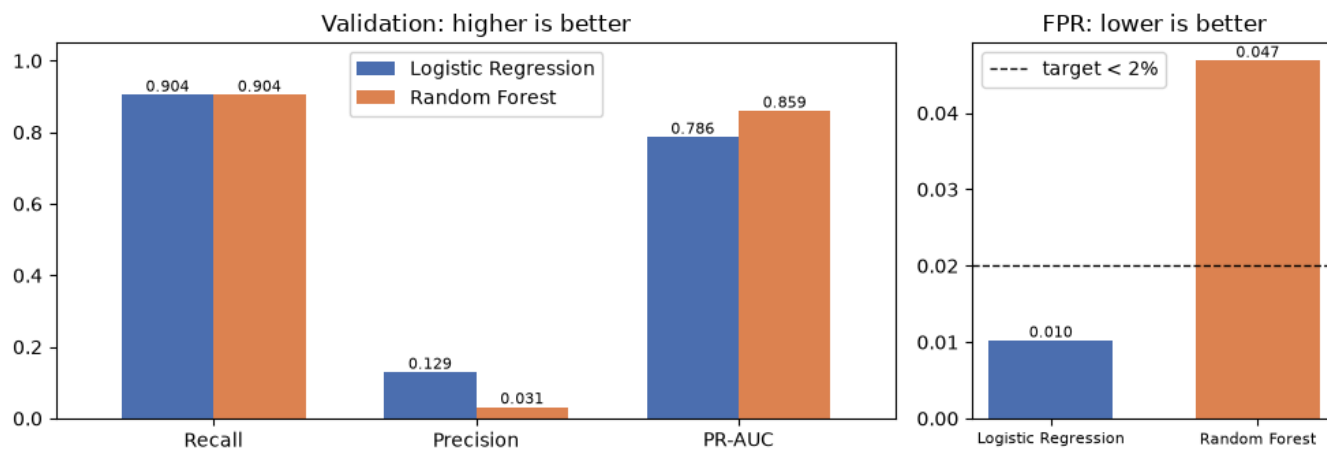
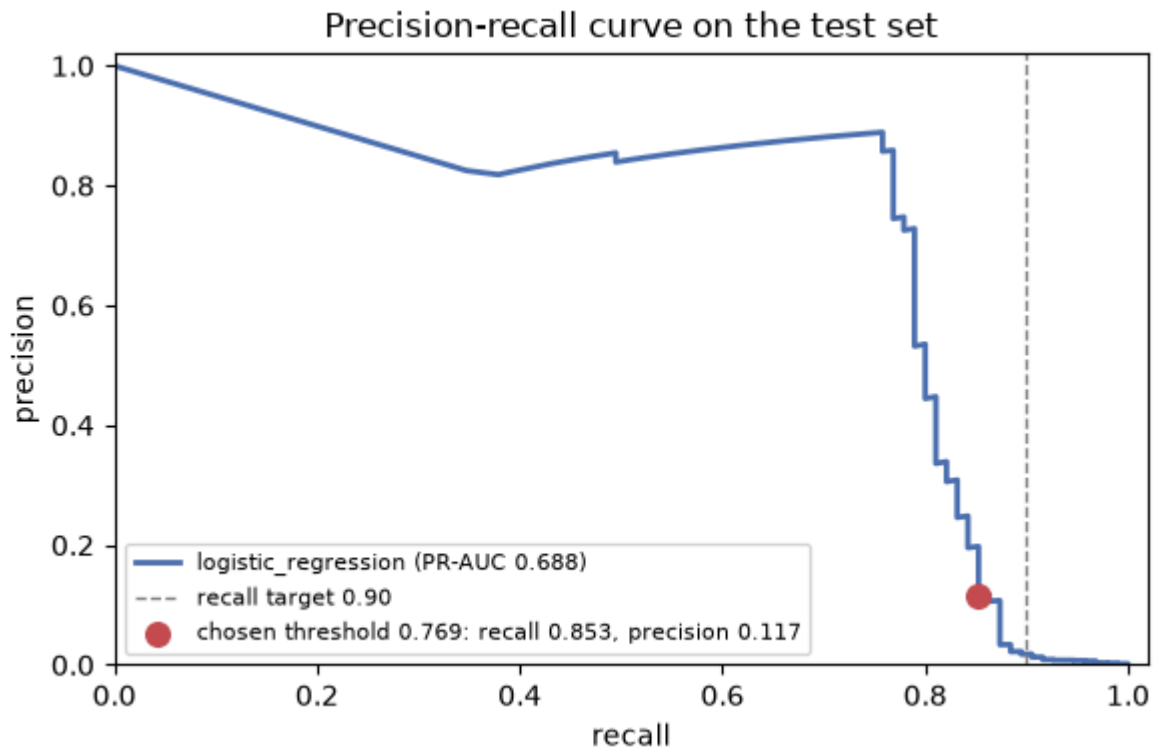


Figure 7 - Precision-recall curve on the test set. This is the saved notebook's frozen native-model curve, not a re-evaluation of the Docker model. The red point uses its validation-selected threshold; the

dashed line is the 90% recall target. Post-hoc analysis of this saved test set gives precision 1.8% at the recall target, with 4,690 genuine transactions flagged. This analysis is not used to tune, serve or promote a model.



7.3 MLflow model registry

Every candidate model is logged as an MLflow run with its parameters (model name, seed, threshold) and its validation metrics. The winning model also gets its test metrics, is registered as fraud-model, and the alias champion is set on this version. The threshold is saved as a run parameter, so the prediction service reads the same value that was chosen during evaluation and it is not written inside the service code. After the run, the Docker registry has fraud-model with Logistic Regression as version 11 with the alias champion, and the stored threshold is 0.7692. At startup the prediction service resolves champion once, loads the fixed numeric model version, and reads the threshold from that same version's run. This prevents a concurrent promotion from mixing a model with another version's threshold. Restart the API after promotion to load it.

7.4 Prediction microservice (FastAPI)

The service has two endpoints. GET /health returns the status and the model version. POST /predict takes the 30 input values and returns request_id, probability, is_fraud, threshold, model_version and latency_ms. The label is true only when the probability is greater than or equal to the threshold. Invalid input gets HTTP 422 and is not logged. If the prediction log cannot be written, the call fails with HTTP 500 and does not continue silently.

```

@app.post("/predict", response_model=Prediction)
def predict(tx: Transaction) -> Prediction:
    """Score one transaction; the label is probability >= threshold and nothing else (BR-001)."""
    start = time.perf_counter()
    # Invalid requests never reach this point, FastAPI has already answered with 422.
    row = pd.DataFrame([tx.model_dump()], columns=config.FEATURES)
    # The saved model scales the input itself, so no separate preprocessing is done here (BR-004).
    probability = float(app.state.model.predict_proba(row)[0, 1])
    is_fraud = probability >= app.state.threshold
    latency_ms = (time.perf_counter() - start) * 1000
    request_id = str(uuid.uuid4()) # unique id so a log row can be matched to a call
    try:
        log_prediction(request_id, probability, is_fraud, latency_ms, app.state.version)
    except OSError as exc:
        # A failed log write is reported as an error, never ignored (BR-008).
        raise HTTPException(status_code=500, detail=f"Prediction log failed: {exc}") from exc
    return Prediction(request_id=request_id, probability=probability, is_fraud=is_fraud,
                      threshold=app.state.threshold, model_version=app.state.version,
                      latency_ms=round(latency_ms, 3))

```

Measured latency: For 100 requests the median is 24.3 ms, the p95 is 32.0 ms and the maximum is 34.1 ms. The target of p95 below 200 ms is met.

7.5 Prediction logging

Every successful prediction is added to predictions_log.csv with the time, request id, probability, label, latency and model version. We do not write the feature values to the log, so no transaction data is stored there.

```

def log_prediction(request_id: str, probability: float, is_fraud: bool, latency_ms: float,
                  model_version: str, path: Path | None = None) -> None:
    """Write one log row; an OSError is raised if the file cannot be written."""
    path = Path(path or config.LOG_PATH)
    # Only result data is logged. The transaction values themselves are not stored.
    row = {"timestamp": datetime.now(timezone.utc).isoformat(), "request_id": request_id,
          "probability": round(probability, 6), "is_fraud": int(is_fraud),
          "latency_ms": round(latency_ms, 3), "model_version": model_version}
    with _lock:
        new_file = not path.exists() or path.stat().st_size == 0 # write the header once
        with path.open("a", newline="") as f: # "a" = append, old rows are kept
            writer = csv.DictWriter(f, fieldnames=FIELDS)
            if new_file:
                writer.writeheader()
            writer.writerow(row)

```

7.6 End-to-end run and test calls

We run the notebook from top to bottom after restarting the runtime. It unzips the dataset from data/creditcard.zip (in Colab it asks for an upload), runs the pipeline, shows the MLflow registry and starts the service in the background. Then it sends one fraud row and one genuine row from the test set, three invalid requests (negative Amount, missing field, unknown field) and 100 requests to measure the latency. At the end it shows the prediction log, runs the automated tests with pytest and, when

Docker is installed, starts the two containers and calls the API through them. The last pytest run shows **32 passed**.

7.7 Containerised deployment (Docker Compose)

The API and model registry run as separate Docker services:

- **prediction-api**: the FastAPI service (container port 8000), using `MLFLOW_TRACKING_URI=http://registry:5000`.
- **registry**: the MLflow tracking and registry service (container port 5000). It runs the SQLite database at `/app/docker-mlflow.db` inside its container. SQLite is embedded in MLflow, not a separate database-server container.

Both services mount the project at `/app`, independent of its Windows, macOS or Linux host path. The database persists as `docker-mlflow.db` in the host project folder; model files persist in `mlruns/`. These files survive stopping and removing the containers. Do not delete them when restarting the Docker workflow. The native Python registry uses a separate `mlflow.db`.

Docker chooses available loopback host ports instead of requiring host ports 5000 and 8000. The final cell reads these ports from Compose and prints the actual API docs and registry URLs. It starts the registry, trains and registers a model inside Docker, recreates the API to load the latest champion, verifies requests, and leaves both services running. They use `restart: unless-stopped` so Docker can restart them after its engine restarts.

This retrains both candidates and requires the real dataset. Docker must be installed and running. Docker output uses UTF-8 decoding with replacement for invalid bytes, avoiding Windows locale decoding failures. If a check fails, services are left available for inspection with `docker compose logs`.

To start or retrain the services manually from the project folder:

```
docker compose up -d --build --wait registry
docker compose run --rm prediction-api python -m pipeline.run_pipeline
docker compose up -d --no-deps --force-recreate --wait prediction-api
docker compose port prediction-api 8000
docker compose port registry 5000
```

The port commands print each service's host address. Open `http://<prediction-api-address>/docs` and `http://<registry-address>`. To resume a previously trained deployment without retraining, use `docker compose up -d --wait`. To stop it explicitly, use `docker compose down`; the database and model files remain in the project folder.

8. Application Screenshots

Verification safeguards: Native results are read from the native registry's current champion, not the metrics file shared with Docker. The frozen model must reproduce its logged validation and test metrics; registry, version and run ID accompany the results. A single version-pinned scorer verifies the current champion, threshold and reproduced metrics and stops on failure. Plotting checks registry, run, version, threshold, row count and checksum and labels the curve as native. The Docker notebook cell checks API/registry/metrics agreement, both sample predictions, and HTTP 422 for negative Amount, missing

features and extra fields. Manual report generation verifies current metrics against the live Docker API and champion's logged run; keep Docker services running when building its report. Older saved notebook outputs are explicitly labelled historical and do not block verified current results. Save the latest notebook outputs to refresh its charts and other recorded evidence. Native and Docker version numbers are independent and must not be substituted for each other. Shared training outputs still require sequential training runs.

Screenshot 1 - Pipeline run. Output of the six-filter pipeline: 284,807 rows read, 1,081 duplicates removed, the 60/20/20 split, both models trained and Logistic Regression selected, with test recall 0.853 and FPR 0.0108.

Pipeline run - six-filter output

```
[1/6 ingest]  rows=284807 columns=31
[2/6 clean]   rows=283726 (removed 1081), fraud=473 (0.167%)
[3/6 features] X=(283726, 30)
[4/6 split]  train=170235 val=56745 test=56746
[5/6 train]   logistic_regression val recall=0.904 precision=0.129 pr_auc=0.7860 threshold=0.7692
[5/6 train]   random_forest      val recall=0.904 precision=0.031 pr_auc=0.8592 threshold=0.0107
[6/6 evaluate] winner=logistic_regression test recall=0.853 precision=0.117 f1=0.206 pr_auc=0.6884 fpr=0.0108
```

Evaluation: Docker registry, version 11

Champion: Logistic Regression | Threshold: 0.769200

Metric	Validation	Test
Recall	0.9043	0.8526
Precision	0.1286	0.1169
PR-AUC	0.7860	0.6884
False-positive rate	0.0102	0.0108

Test recall target >= 90%: NOT MET. False-positive rate target < 2%: PASS.

Current metrics verified against live Docker; saved stage output is historical.

Screenshot 2 - MLflow. Live Docker registry page of fraud-model. Version 11 is registered and has the alias champion.

mlflow

3.17.0

GenAIModel training

Home

Experiments

Prompts

MCP registryBeta

AI GatewayNew

Docs

Settings

Registered Models >

fraud-model

Created Time: 10/08/2026, 10:21:26 AM

Last Modified: 10/08/2026, 01:04:07 PM

> DescriptionEdit

> Tags

> VersionsCompare

New model registry UI

Version	Registered at	Created by	Tags	Aliases	Description
Version 11	10/08/2026, 01:04:07 PM		Add	@ champion	
Version 10	10/08/2026, 12:04:55 PM		Add	Add	
Version 9	10/08/2026, 11:52:25 AM		Add	Add	
Version 8	10/08/2026, 11:03:26 AM		Add	Add	
Version 7	10/08/2026, 10:55:28 AM		Add	Add	
Version 6	10/08/2026, 10:39:45 AM		Add	Add	
Version 5	10/08/2026, 10:36:17 AM		Add	Add	
Version 4	10/08/2026, 10:29:56 AM		Add	Add	
Version 3	10/08/2026, 10:28:02 AM		Add	Add	
Version 2	10/08/2026, 10:26:50 AM		Add	Add	
Version 1	10/08/2026, 10:21:26 AM		Add	Add	

< Previous

Next >

25 / page

Screenshot 3 - FastAPI. The live Docker API /docs page with GET /health and POST /predict, request fields (Time, V1 to V28, Amount), and response fields.

Credit Card Fraud Detection Service

0.1.0

OAS 3.1

/openapi.json

default

GET

/health

Health

Report that the service is up and which model version it serves.

Parameters

Try it out

No parameters

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value

Schema

```
{
  "additionalProp1": {}
}
```

POST

/predict

Predict

Score one transaction; the label is probability >= threshold and nothing else (BR-001).

Parameters

Try it out

No parameters

Request body

required

application/json

Example Value

Schema

```
{
  "Time": 0,
  "V1": 0,
  "V2": 0,
  "V3": 0,
  "V4": 0,
  "V5": 0,
  "V6": 0,
  "V7": 0,
  "V8": 0,
  "V9": 0,
  "V10": 0,
  "V11": 0,
  "V12": 0,
  "V13": 0,
  "V14": 0,
  "V15": 0,
  "V16": 0,
  "V17": 0,
  "V18": 0,
  "V19": 0,
  "V20": 0,
  "V21": 0,
  "V22": 0,
  "V23": 0,
  "V24": 0,
  "V25": 0,
  "V26": 0,
  "V27": 0,
  "V28": 0,
  "Amount": 0
}
```

Screenshot 4 - Fraud test call. A saved native notebook request, serving model version 1. The fraud probability is 1.0000, is_fraud is true and the answer came in 56 ms.

POST /predict - fraud transaction

```
--- fraud transaction (HTTP 200)
{
  "request_id": "a7eb6b4c-0790-4138-8246-6c14ad523daf",
  "probability": 0.9999999998115436,
  "is_fraud": true,
  "threshold": 0.7691997375810586,
  "model_version": "1",
  "latency_ms": 55.725
}
```

Screenshot 5 - Genuine test call. A saved native notebook request, serving model version 1. The probability is 0.028, below the threshold 0.7692, so is_fraud is false.

POST /predict - genuine transaction

```
--- genuine transaction (HTTP 200)
{
  "request_id": "83fe0c62-df31-4d27-8473-e5168cea57a6",
  "probability": 0.027732498227629385,
  "is_fraud": false,
  "threshold": 0.7691997375810586,
  "model_version": "1",
  "latency_ms": 3.3
}
```

Screenshot 6 - Prediction log. The prediction log after the test calls and the 100 latency requests: 102 rows, each with timestamp, request id, probability, label, latency and model version.

Prediction log (predictions_log.csv)

```
Rows logged: 102
```

	timestamp	request_id	probability	is_fraud	latency_ms	model_version
97	2026-10-08T05:09:10.592762+00:00	19a1a5f4-fe4e-45c3-a562-39567a5e9b35	0.027732	0	3.380	1
98	2026-10-08T05:09:10.624757+00:00	b5e5e81a-858f-4316-8b90-0ffd047b488f	0.027732	0	3.440	1
99	2026-10-08T05:09:10.655192+00:00	34cc6e78-c307-498c-a267-f8733c1a360c	0.027732	0	2.657	1
100	2026-10-08T05:09:10.684940+00:00	5ccb8231-f498-440f-aff7-e6fa80ac82d9	0.027732	0	2.710	1
101	2026-10-08T05:09:10.717799+00:00	f5cd46ef-3a39-405d-a3ac-272bdd41a8c8	0.027732	0	4.390	1

Screenshot 7 - Docker Compose. Live Compose status and live registry verification: registry (container port 5000) and prediction-api (container port 8000), published on dynamically assigned

localhost ports. The Docker champion is version 11; services remain running until manually stopped.

Docker Compose: live status and registry verification

NAME	IMAGE	STATUS	PORTS
assigenment-1-prediction-api-1	fraud-detection:1.0	Up About an hour (healthy)	127.0.0.1:63996->8000/tcp
assigenment-1-registry-1	sha256:563f3f407a951e3023fb62b4a00a06c88aab034fe2eaa5b9d299c520732fc76c	Up 4 hours (healthy)	127.0.0.1:58706->5000/tcp

```
Live Docker champion version: 11
MLflow run: 459c82fa0aa94a05afca63ef0e3828b7
Health: {'status': 'ok', 'model': 'fraud-model', 'model_version': '11', 'threshold': 0.7691997772934724}
Saved Docker notebook version: 6; retained as historical evidence.
Current evaluation metrics verified against the live MLflow run.
```

Screenshot 8 - Docker database. Read-only SQLite inspection executed inside the registry container, showing the database file, tables, and model-version count. /app/docker-mlflow.db persists in the bind-mounted project folder; SQLite is embedded in the registry, not a separate database-server container.

Docker registry: persistent SQLite database (read-only inspection)

```
{
  "database": "/app/docker-mlflow.db",
  "size_bytes": 1196032,
  "tables": [
    "experiments",
    "model_versions",
    "registered_model_aliases",
    "registered_models",
    "runs"
  ],
  "model_versions": 11
}
```

9. Conclusion

In this assignment we built a fraud detection system with a pipe-and-filter training pipeline, an MLflow registry and a FastAPI microservice that serves the champion model, with prediction logging and 32 automated tests. With Docker Compose the registry and the prediction API also run as two separate containers. Against our goals: false-positive rate (1.08% against 2%) is met, latency (p95 32.0 ms against 200 ms) is met, and the test recall is 85.3% against the 90% target, so that goal is not met. The estimated fraud loss reduction is 82.4% against the 30% target (met). From this work we learned that with so few frauds in each split the recall on the test set can differ from the validation recall, that the model for serving should be chosen on validation results only, and that a registry alias keeps the service independent of the model version. If we had more time we would try more tuning on validation, cross-validation for the threshold, and more fraud examples.